

NeoVim Cheat - Normal Mode

~ toggle case	! 1 2	@. play macro 2	# find (rev) identifier 2	\$ eol 2	% goto match 2	^ "soft" bol 2	& repeat :s 2	* find identifier 2	("soft" bol 2	) end sentence	_ "soft" bol down	+ next line	Backsp delete left char
. goto mark	1	2	3	4	5	6	7	8	9	0 "hard" bol	- previous line	= auto format 9	
Q play last macro	W next WORD	E end WORD	R replace mode	T. hop char full screen	Y yank to eol	U undo line	I insert at bol	O open above	P paste before 1	{ begin paragraph	} end paragraph	bol / go to column	
q. record macro	w next word	e end word	r. replace char	t. hop char cur buffer	y yank 1,8 9	u undo	i insert mode	o open below	p paste after 1	[. previous 3	]. next 3	\ resume telescope	
Caps = Esc normal mode	A append at eol	S delete line	D delete to eol	F. find char backward	G eof/ go/line	H screen top	J join lines	K help / doc	L screen bottom	: command line	" choose register 1	Enter change word	
	a append 7	s delete char	d delete 1,7 8,9	f. find char forward	g. extra cmds 5	h	j	k	l	; hop word full screen	' goto mark bol		
Z	X back-space	C delete to eol	V visual lines	B back WORD	N previous (find)	M screen middle	< un-indent 9	> indent 9	? find (rev)				
Z. scroll cmds 4	X delete char	c change 1,7 8,9	v visual mode	b back word	n next (find)	m. set mark 6	,	. repeat cmd	/ find				

bol = beginning of line
eol = end of line,

bol = beginning of line
eol = end of line,

motion	moves the cursor, or defines the range for an operator
command	direct action command, if red it enters insert mode
operator	requires a motion afterwards, operates between cursor & destination
extra	special functions, requires extra input
q.	commands with a dot need a char argument afterwards

main command line commands ('ex'):

```

:w (save), :q (quit) :wq (save&quit), :q! (quit w/o saving)
:f (open file f), :vsp f (open file in vertical split)
:%s/x/y/gc (replace 'x' by 'y' filewide, but ask),
:%s/\s+$/ / (remove all trailing spaces),
:h (help in neovim), :new (new file in neovim)
:[number] (go to line [number])
:reg (show content of registers)

```

visual mode:

- operator use selection instead of motion
- o: exchange cursor position with start of selection
- J: join all selected lines
- S:: add surrounding delimiter

notes:

- (1) use "x before a yank/paste/del command to use that register ('clipboard') (x=a..z.*)" (e.g.: "ay\$ to copy rest of line to register 'a'")
 - (2) type in a number before any action to repeat it that number of times (e.g.: 2p, d2w, 5i, d4j)
 - (3) [m /]m - go to previous / next method start [M /]M - go to previous / next method end [t /]t - go to previous / next TODO or NOTE [e /]e - go to previous / next error or warning [g /]g - go to previous / next git hunk
 - (4) zz: scroll cursor to center, zt: top, zb:bottom
 - (5) gg: top of file, gi: go to last insertion and switch to insert mode gv: reselect last visual selection gm: middle of screen line, gM: middle of line ge: previous end of word gn / gN: go to next/previous find and select gt / gT: go to next/previous tab
 - (6) m; - set next available mark (alphabetical) m; - toggle mark at current line m] and m[- go to next/previous mark dmx and dm<space> - delete mark x / delete all marks
 - (7) ci(- change inside brackets; also),[,.{,<,>,"',, w,s,p ca(- change around brackets di(, da(, yi(, ya(, vi(, va(for delete, yank, select inside/around
 - (8) ys[motion]{new} - add surrounding {new} to {motion} cs[old]{new} - change surrounding {old} with {new} ds[old] - delete surrounding {old} yss{new} - add surrounding {new} to line [space]b - change brackets (same as csb) {new/old} = (,)[,],{,<,>,"',, s (any of those) b (brackets:),,],>), q (quotes: ',',, t (html-tags), f (function), i (individual start and end)
 - (9) duplicate operator to act on current line (dd = delete line, yy = yank line, >> = indent line)